

Informe Consolidado de Evaluación y Escaneo de Vulnerabilidades

Proyecto: Gemini-Interpreter / KogniTerm
Ubicación: /home/gato/Proyectos/Gemini-Interpreter
Versión de Proyecto: 1.2.1
Fecha de Evaluación: 10 de Agosto de 2026
Evaluador: Agente KogniTerm (Vulnerability Scanning & Assessment)

Resumen Ejecutivo

Se ha realizado una auditoría estática y dinámica de seguridad sobre la base de código Python (kogniterm), sus dependencias (pyproject.toml , setup.py) y sus configuraciones de servidor REST y WebSocket API.

Nivel de Riesgo	Cantidad	Vulnerabilidades Principales
CRITICAL	1	Bypass de Autenticación en WebSocket (ws_chat)
HIGH	1	Inyección de comandos en subprocess con shell=True
MEDIUM	3	Binding de API por defecto en 0.0.0.0 , Exención de Auth por IP Local, Peticiones HTTP sin Timeout
LOW	2	Silenciamiento de Excepciones (except: pass), Dependencias sin versión fija (pyproject.toml)

Hallazgos Detallados de Seguridad

1. **CRITICAL: Bypass de Autenticación en WebSocket (/ws/chat y /ws/{session_id})**

- **Archivo:** `kogniterm/server/app.py` (Líneas 1453-1457 y 265)
- **CVSS v3.1 Score: 9.8 (Critical)** AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
- **CWE:** [CWE-287: Improper Authentication](#)

Descripción

En `require_token` (middleware de la API FastAPI), las solicitudes de tipo `websocket` son omitidas de la verificación global:

```
if request is None or request.scope.get("type") == "websocket":  
    return
```

Posteriormente, en el handler del WebSocket (`websocket_chat`), la validación del token de la URL es:

```
token = websocket.query_params.get("token")  
if token and not secrets.compare_digest(token, API_TOKEN):  
    await websocket.close(...)  
    return
```

Falló la verificación: Si un cliente se conecta **sin proporcionar el parámetro token** (es decir, `token is None`), la condición `if token` evalúa como `False`, omitiendo el chequeo y permitiendo el acceso completo por WebSocket a cualquier cliente no autenticado.

Impacto

Un atacante remoto puede conectarse a `ws://<HOST>:8765/ws/chat` sin token y enviar mensajes al agente para ejecutar código o herramientas en el sistema del usuario con privilegios totales.

Remediación Sugerida

Modificar la condición para **exigir obligatoriamente** la presencia del token:

```
token = websocket.query_params.get("token")
if not token or not secrets.compare_digest(token, API_TOKEN):
    logger.warning("[WS] Conexión rechazada: token faltante o inválido.")
    await websocket.close(code=status.WS_1008_POLICY_VIOLATION)
    return
```

2. HIGH: Inyección / Ejecución de Comandos en Subprocesos (`shell=True`)

- **Archivos:** `kogniterm/core/background_task_manager.py` (Línea 132), `kogniterm/skills/bundled/execute-command/scripts/tool.py` (Línea 221)
- **CVSS v3.1 Score: 8.8 (High)**
`AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`
- **CWE / Bandit:** [CWE-78 / B602: Subprocess call with shell=True](#)

Descripción

Bandit identificó llamadas a `subprocess.Popen` y `subprocess.run` pasando `shell=True` junto con comandos pasados como cadenas de texto. Si los argumentos contienen metacaracteres de shell (ej: `;`, `&&`, `|`, ```), un comando malicioso puede encadenar la ejecución de otros comandos arbitrarios.

Remediación Sugerida

Utilizar `shlex.split()` y llamar a `subprocess.Popen` en forma de lista de argumentos con `shell=False`:

```
cmd_args = shlex.split(command)
process = subprocess.Popen(cmd_args, shell=False, ...)
```

3. MEDIUM: Escucha por Defecto en Todas las Interfaces de Red (0.0.0.0)

- **Archivos:** `kogniterm/server/app.py` (Línea 1594), `kogniterm/server/config.py` (Línea 30)
- **CVSS v3.1 Score: 6.5 (Medium)** AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
- **CWE / Bandit:** [CWE-605 / B104: Hardcoded bind all interfaces](#)

Descripción

El servidor de FastAPI por defecto se enlaza a `0.0.0.0` (`host: str = "0.0.0.0"`), lo que abre el puerto 8765 a cualquier interfaz de red física o virtual (Wi-Fi, Ethernet, VPN) disponible en la máquina.

Remediación Sugerida

Modificar la configuración por defecto para escuchar únicamente en la interfaz local (`127.0.0.1`), permitiendo `0.0.0.0` solo mediante bandera explícita `--host 0.0.0.0` si el usuario lo requiere conscientemente:

```
class ServerSettings(BaseModel):  
    . host: str = "127.0.0.1" # Escucha segura local por defecto  
      port: int = 8765
```

4. MEDIUM: Bypassing de Autenticación por IP Loopback Local (127.0.0.1)

- **Archivo:** `kogniterm/server/app.py` (Línea 273)
- **CVSS v3.1 Score: 5.3 (Medium)** AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
- **CWE:** [CWE-290: Authentication Bypass by Spoofing](#)

Descripción

La función `require_token` confía en `request.client.host` :

```
client_host = request.client.host if request.client else ""
if client_host in ("127.0.0.1", "::1", "localhost"):
    return
```

Si el servidor se coloca tras un proxy inverso (como Nginx o Cloudflare) sin reescribir `X-Forwarded-For`, o si se explota una vulnerabilidad SSRF o DNS Rebinding desde una aplicación local, cualquier petición se considerará proveniente de `127.0.0.1` y la autenticación API será completamente ignorada.

Remediación Sugerida

Exigir el `API_TOKEN` en todas las llamadas REST y WebSocket independientemente de la IP origen, salvo en endpoints explícitamente públicos (`/health`).

5. MEDIUM: Ausencia de Timeout en Clientes HTTP (`requests`)

- **Archivo:** `kogniterm/terminal/api_client.py` (Líneas 18 y 31)
- **CVSS v3.1 Score: 5.3 (Medium)** `AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`
- **Bandit:** [B113: Request without timeout](#)

Descripción

Las llamadas HTTP con la librería `requests` no especifican el parámetro `timeout`. Si el servidor remoto o local deja la conexión colgada, la CLI de KogniTerm se congelará indefinidamente.

Remediación Sugerida

Establecer siempre un timeout explícito:

```
resp = requests.get(f"{API_BASE_URL}/config/llm", timeout=10.0)
```

6. LOW: Silenciamiento Indiscriminado de Excepciones (`try...except: pass`)

- **Archivos:** Múltiples módulos (`tui_app.py` , `llm_service.py` , `meta_command_processor.py`)
- **Bandit:** [B110: Try, Except, Pass detected](#) (251 ocurrencias)

Descripción

Múltiples bloques capturan cualquier excepción silenciando el error con `pass` . Aunque en elementos UI evita crasheos visuales, en la lógica de negocio puede enmascarar fallos de permisos, IO o errores de inicialización.

Remediación Sugerida

Registrar los errores con `logger.debug(...)` o `logger.warning(...)` en lugar de `pass` .

7. LOW: Dependencias de Python sin Anclaje de Versión (`Unpinned Dependencies`)

- **Archivo:** `pyproject.toml`
- **CWE:** [CWE-1104: Use of Unmaintained or Vulnerable Third-Party Components](#)

Descripción

La mayoría de las dependencias (`fastapi` , `uvicorn` , `litellm` , `requests` , `pygithub` , etc.) no tienen límites ni fijación de versión estricta en `pyproject.toml` .

Remediación Sugerida

Especificar límites de versión compatibles (ej: `fastapi>=0.110.0,<1.0.0`) y mantener actualizado el archivo `uv.lock` .

Plan de Mitigación Recomendado (Roadmap de Remediación)

1. Inmediato (24 horas):

2. Corregir la validación de `token` en `WebSocket (kogniterm/server/app.py:1453)`.
3. Cambiar la IP de escucha por defecto de `0.0.0.0` a `127.0.0.1`.

4. Corto Plazo (1 semana):

5. Eliminar `shell=True` en `background_task_manager.py` y en la skill `execute-command`.
6. Exigir `token` en `require_token` eliminando la exención incondicional por IP `127.0.0.1`.

7. Mantenimiento Continuo:

8. Añadir `timeout` a todas las llamadas `requests`.
9. Reemplazar bloques `except: pass` por registro de logs estructurado.